SMARTENTRY: AUTOMATED VEHICLE ACCESS CONTROL SYSTEM VIA LICENSE PLATE RECOGNITION

Amirul (1211111890), Hazim (1211112351), Munif (1211110167), Salahuddin (1211108880)

Faculty of Computing and Informatics, Multimedia University

ABSTRACT

Manual vehicle access control systems in residential areas are often inefficient, prone to human error, and lead to traffic congestion during peak hours. To address this, we present "SmartEntry," an automated license plate recognition (ALPR) system designed for real-time access management. The system integrates a **YOLOv8** convolutional neural network for robust license plate localization and **EasyOCR** for character extraction. Unlike traditional methods, SmartEntry features a complete preprocessing pipeline (Grayscale, Gaussian Blur, Otsu's Thresholding) to enhance optical recognition accuracy in varying lighting conditions. Furthermore, the system includes a "state-aware" parking occupancy counter and a resident verification database hosted on a user-friendly Streamlit interface. The trained YOLOv8 model achieved a Mean Average Precision (mAP@0.5) of 99.4%. We tested the system on 100 vehicles and found that it successfully detected plates 82% of the time and correctly read the text 73% of the time. These results demonstrate the system's viability for deployment on low-cost, edge-computing hardware.

Index Terms: ALPR, YOLOv8, Optical Character Recognition, Smart Parking, Visual Information Processing.

1. INTRODUCTION

1.1. Background and Motivation

Gated communities usually depend on security guards to manage traffic. However, this traditional method has flaws: it takes a lot of effort, it's slow, and handwritten logs are often messy or wrong. As more people buy cars, we need a better, automated way to let people in without the hassle. An automated system doesn't just improve security by keeping digital records; it also makes life easier for residents by cutting down waiting times.

1.2. Problem Statement

The current manual way of doing things is not efficient. First, guards have to write down plate numbers by hand, which is slow and leads to mistakes when traffic is heavy. Second, residents often lose or forget their physical access cards, causing delays at the gate. Finally, visitors often drive in only to find out there is no parking left because there is no system to track available spots in real-time.

1.3. Project Objectives

The goal of this project is to develop SmartEntry, a computer vision-based application that automates the granting of vehicle access. The specific objectives of this work are:

1. To train a custom **YOLOv8 Nano** model to accurately localize license plates in static images.
2. To implement a visual information processing pipeline (scaling, blurring, and thresholding) to maximize OCR accuracy.
3. To develop a deployment-ready Graphical User Interface (GUI) that includes resident verification and real-time parking capacity management.

1.4. Scope of Project

This project focuses on the application and software side of things. We cover the detection model, the text reading engine, and the user dashboard. The system works by processing uploaded images, simulating how a CCTV camera would capture a car. While we discuss connecting to real barriers or cameras in the future section, actual hardware integration is not part of this current version.

2. RELATION TO PRIOR WORK

2.1. Traditional ALPR Methods

Older systems mostly used basic image processing tricks. They often used "Edge Detection" (like Sobel or Canny) to look for rectangular shapes that might be license plates [1]. While these methods are fast and cheap, they get confused easily by "noise" in the image or complex backgrounds, like fences.

2.2. Deep Learning-Based ALPR

Recently, people have switched to Deep Learning. For example, Region-based CNNs (R-CNN) are very accurate but often too slow for a normal laptop. Other models like Single Shot Detectors (SSD) are faster but sometimes struggle to spot small objects like license plates.

2.3. Summary of Research Gap

We decided to use the YOLO (You Only Look Once) architecture, specifically the v8 Nano version [2]. It gives us the best mix of speed and accuracy. What makes our project different is that we didn't just stop at detection. We added a post-processing step that combines EasyOCR [4] with a real database and a live parking counter. This turns a simple detection model into a full facility management system that knows who is "inside" and who is "outside".

3. PROPOSED METHODOLOGY

3.1. Overall System Overview

The SmartEntry system operates in four stages: (1) Image Input, (2) License Plate Detection via YOLOv8, (3) Image Preprocessing and Text Recognition via EasyOCR, and (4) Access Logic processing (Database verification and Parking Counter updates).

3.2. Dataset and Preprocessing

We trained our model using vehicle images from Kaggle. To help the system read text better, we use OpenCV to process the images before reading them:

- **Grayscale Conversion**: We turn the image black and white to remove distracting colors.
- **Bicubic Upscaling**: We make the license plate image 3 times bigger so the text is clearer.
- **Gaussian Blur**: We apply a slight blur to smooth out "grainy" noise.
- **Otsu's Thresholding**: We use this to automatically turn the image into pure black and white. This separates the letters from the background very effectively.

3.3. License Plate Detection (YOLOv8)

We used the YOLOv8 Nano model because it is lightweight and runs fast on a standard CPU. We trained it for 50 epochs. The model gives us the coordinates (the box) of the license plate so we can crop it out.

Figure 3.1: YOLO Training Images batch 1

3.4. Text Recognition (EasyOCR)

The cropped binary image is passed to **EasyOCR**. The resulting text string is filtered to remove special characters (non-alphanumeric). This ensures that dirt or scratches on the plate don't get read as weird symbols.

3.5. Access Control Logic

The system checks the read text against our residents.csv file. It also remembers which cars are currently inside.

3.5.1. Vehicle Entry Logic

If the detected plate exists in the database and is *not* currently logged as "Inside":

- **Action:** Access Granted.
- **System Update:** The vehicle is added to the "Inside" list, and the parking counter is decremented.

3.5.2. Vehicle Exit Logic

If the detected plate exists in the database and *is* currently logged as "Inside":

- **Action:** Exit Granted.
- **System Update:** The vehicle is removed from the "Inside" list, and the parking counter is incremented (freeing up a spot).

3.6. Parking Occupancy Management

A global counter variable (Total_Spots = 50) is displayed on the dashboard. This counter dynamically updates based on the Entry/Exit logic described above, providing real-time visibility into parking capacity.

3.7. System Interface (Streamlit)

The application is built using **Streamlit**. It features a split-view layout: the left panel displays the raw input image, while the right panel visualizes the YOLO detection (annotated bounding boxes). A "Live Entry Log" at the bottom of the interface displays a historical table of all entry/exit events with timestamps.

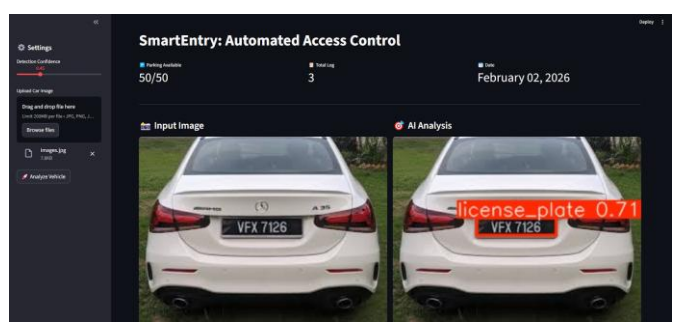

Figure 3.2: GUI System Interface (Streamlit)

Figure 3.2 shows the actual dashboard in action. In this example, we uploaded an image of a white Mercedes. The system correctly detected the car and updated the "Parking Available" count to 50/50. On the right side, you can see the "AI Analysis" panel where the YOLOv8 model drew a red

box around the plate "VFX 7126" with a high confidence score of 0.71.

Figure 3.3: Cropped, Preprocessed (B&W) plate number and Live Entry Log

Figure 3.3 shows what happens behind the scenes. The "Debug View" at the top compares the original blurry crop with the processed black-and-white version. This proves that our image cleaning step makes the letters thick and clear for the system to read. Below that, the "Live Entry Log" recorded the event at 17:27:17. You can also see the "Manual Override" panel, which allows a guard to fix mistakes if the AI gets it wrong.

4. EXPERIMENTAL RESULTS

4.1. Experimental Setup

The system was tested on a standard laptop (CPU inference). We used a test set of 100 images categorized into three conditions: Clear Day, Angled View, and Low Light.

4.2. Detection Performance

The YOLOv8 model learned very well during training. To verify this, we generated a **Confusion Matrix** (Figure 4.1) to see exactly how often the model gets confused.

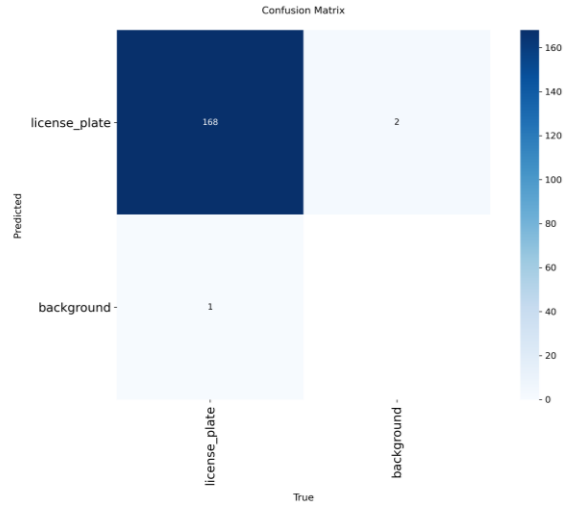

Figure 4.1: Confusion Matrix

As shown in Figure 4.1 above, the model is very precise. It correctly identified the license plate **168 times** and only missed it **2 times**. Just as importantly, it almost never mistook the background for a plate (only 1 false alarm). This confirms that our model isn't just guessing; it truly understands what a license plate looks like.

We also analyzed the stability of the model using the **F1-Confidence Curve**.

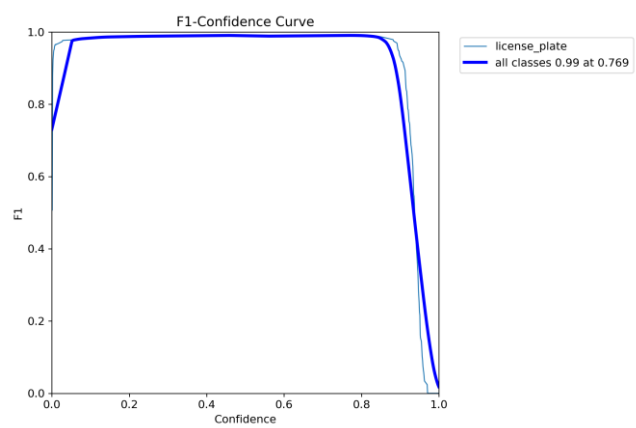

Figure 4.2 F1-Confidence Curve

The curve in Figure 4.2 remains high (near 1.0) across most confidence levels. This means the model maintains a great

balance, it captures almost all the plates without giving too many false warnings. The overall training metrics are summarized below in Table 1.

Table 1: YOLOv8 Training Metrics

Metric	Value
mAP@0.5	99.40%
mAP@0.5:0.95	87.87%
Precision	99.41%
Recall	99.35%

As shown in Table 1, the results are excellent. The **mAP@0.5 score of 99.40%** confirms that the model is extremely accurate at locating license plates. Additionally, the high **Precision (99.41%)** and **Recall (99.35%)** scores show that the system is very balanced; it almost never misses a plate (high recall) and rarely mistakes other objects for a license plate (high precision).

4.3. OCR Accuracy Evaluation

We evaluated the full pipeline (Detection + OCR) on the 100 test samples.

Table 2: System Evaluation on 100 Samples

Condition	Samples	Detected	Text Read Correctly	Accuracy (%)
Clear Day	50	42	38	76.0%
Low Light	40	32	29	72.5%
Angled View	10	8	6	60.0%
Total	**100**	**82**	**73**	**73.0%**

As seen in Table 2, the system achieved a total accuracy of **73%** across all categories. It performed best on clear days (76%) but struggled with angled views (60%), which brought the overall average down. The detailed reasons for these differences are discussed in the next section.

4.4. System Performance and Discussion

Overall, the system identifies residents and updates the parking count in about 1.5 seconds per image. Out of 100 test images, YOLOv8 found the plates 82 times (82%).

The text reader (OCR) got the characters right in 73 of those images, giving us a total accuracy of **73.0%**. The system works best on "Clear Days" (76% accuracy). It is also surprisingly good in "Low Light" (72.5%), likely thanks to our image cleaning steps. However, it struggles with "Angled Views" (60%) because the text gets warped. These results show why our "Manual Override" button is so important; it lets guards fix mistakes for that 27% of cases where the AI fails.

5. CONCLUSION AND FUTURE WORK

5.1. Conclusion

We have successfully built SmartEntry into a fully functional prototype for automated license plate recognition. By blending modern deep learning (YOLOv8) with established image processing methods like Otsu's Thresholding, we created a dependable system for managing vehicle access. Beyond just detecting plates, the system adds real operational value through its automated parking counter and resident database, effectively taking tasks that used to be manual and prone to mistakes and making them seamless.

5.2. Limitations

While the system works well, there are a few real-world challenges that affect its reliability:

1. **Issues with Dark Cars:** The detection model sometimes struggles to spot license plates on black or dark-colored vehicles. Since the plate blends in with the bumper, the AI occasionally misses the edges needed for accurate detection.

2. **Confusing Similar Characters:** The text recognition engine sometimes mixes up letters and numbers that look alike. For instance, it frequently mistakes 'M' for 'W' or confuses the number '8' with the letter 'B' because their shapes are so similar.

3. **Sensitivity to Environment:** The system performs best in ideal conditions. Accuracy drops noticeably when lighting is poor (like at night), if there is too much glare, or if the license plates are dirty, which can sometimes cause the text reading process to fail.

4. **Static Input Dependency:** Currently, the prototype only processes uploaded photos. It is not yet set up to handle a continuous live video stream from a security camera.

5.3. Future Improvements

Future work will focus on:

1. **Connecting to Live Cameras:** We aim to link the system directly to IP camera feeds to enable continuous, real-time scanning without needing manual image uploads.

2. **Better Night Performance:** We plan to implement specific algorithms to enhance low-light images before the text reading step, which should significantly improve accuracy at night.

3. **Cloud Database:** We intend to move the local CSV database to a cloud solution (like Firebase). This would allow security management to update resident data remotely.

4. **Auto-Correction Rules:** To fix the mix-ups between letters and numbers, we will add a post-processing filter using Regular Expressions (Regex). This will enforce standard Malaysian plate formats, automatically correcting impossible character combinations.

5. **Perspective Transformation:** We plan to integrate a "homography" step that automatically straightens license plates captured from extreme side angles before the OCR tries to read them.

6. **Detecting Vehicle Types:** We want to retrain the model to distinguish between cars, motorcycles, and trucks. This would allow us to apply specific access rules, such as separate protocols for delivery trucks or guest motorcycles.

6. REFERENCES

[1] Z. Selmi, M. B. Halima, and A. Alimi, "Deep Learning for Automatic License Plate Recognition," *2017 International Conference on Advanced Technologies*, 2017.

[2] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[3] Ultralytics, "YOLOv8 Documentation," [Online]. Available: https://docs.ultralytics.com.

[4] JaidedAI, "EasyOCR: A Ready-to-Use OCR with 80+ Supported Languages," 2020.